PROJECT OVERVIEW · 2026

TWW3 Tournament Platform

Discord-authenticated tournament platform for Total War: Warhammer 3 — Single-Elim, Swiss, Round-Robin, Captain's-Mode-Draft, ELO-Leaderboard, Live-Bracket-Realtime.

STAND

M5 abgeschlossen · Production-Launch-Ready

STACK

Fastify 5 · Prisma 7 · Vite 6 · React 19

TESTS

301 Backend · 60 Frontend · 23+ E2E · 12 Scraper

TIMELINE

Greenfield 2026-05-12 → M5 2026-05-13

generated · 2026-05-13 · solo-dev · ai-first

01 Executive Summary

> Eine Single-Tenant-Plattform für ein TWW3-Tournament-Ökosystem. Greenfield-Implementierung in einer Woche, AI-first Solo-Dev, fünf abgeschlossene Milestones, Production-Launch-Ready.

Was kann die Plattform?

- **Discord-OAuth-Login** mit Rollen-System (USER / ORGANIZER / MODERATOR / ADMIN)
- **Tournament-Formate:** Single-Elimination, Swiss-System mit Rematch-Avoidance, Round-Robin
- **Live-Bracket** mit Socket.IO — Match-Results pushen real-time an alle Viewers
- **Captain's-Mode-Draft** mit Faction-Picks/Bans, Reveal-Phasen, Timer und Spectator-Maskierung
- **ELO-Ranking** (Multi-Player Performance-Rating, zero-sum, K=32/48)
- **Faction-Meta-Stats** mit 24×24-Matchup-Heatmap und täglichen Snapshots
- **Admin-Panel**: Audit-Log, Stats-Dashboard, Ban-Management, Preset-Promotion
- **Army-List-Upload** (TXT/PDF, max 5 MB) mit naivem Parser für Lord/Heroes/Units

Test & Quality

301	60	23+	5
BACKEND TESTS	FRONTEND TESTS	E2E SPECS	MILESTONES

Architektur in einem Satz

pnpm-Monorepo mit zwei Apps (`apps/backend` , `apps/frontend`), zwei Shared-Packages (`@tww3/db` , `@tww3/types`), einer E2E-Suite (`apps/e2e`) und einem CLI-Scraper (`scraper/`); Backend ist eine Fastify-5-App mit driver-adapter-Prisma, Socket.IO direkt an `fastify.server` und ioredis-Adapter; Frontend ist Vite-6 + React-19 + TanStack Router/Query + Tailwind-4 CSS-first.

INHALT DIESES DOKUMENTS

08 Production-Launch-TODOs & Commands

02 Tech-Stack & Versionen

> Konservativ aktuelle Major-Versionen, validiert beim Greenfield-Start 2026-05-12. Node ≥ 22, pnpm 9, ESM überall.

Runtime & Toolchain

BEREICH	TOOL	VERSION	ZWECK
Runtime	`Node.js`	≥ 22	ESM-only, native fetch
Package Manager	`pnpm`	9.15.0	Workspaces, content-addressable store
Build-Orchestrator	`Turborepo`	2.x	Task-Pipeline, Caching
Language	`TypeScript`	5.x strict	überall
Linting	`ESLint`	flat-config	CI-grünes Lint-Step
Formatting	`Prettier`	3.x	Repo-weit

Backend

LAYER	LIBRARY	VERSION	NOTIZ
HTTP-Server	`Fastify`	5	Plugin-System, mehrstufige Hooks
ORM	`Prisma`	7	**driver-adapter** `@prisma/adapter-pg`
Realtime	`Socket.IO`	4	Direkt an `fastify.server`, kein Plugin
Cache / Pub-Sub	`ioredis`	5	3 Instanzen: `redis`, `redisPub`, `redisSub`
GraphQL	`mercurius`	—	Parallel zu REST, `/graphql`
Cron	`node-cron`	—	Täglicher Faction-Snapshot 00:05 UTC
Pairing-Algos	`tournament-pairings`	2.0.1	SingleElim, Swiss, RoundRobin
Multipart	`@fastify/multipart`	—	Army-List-Upload
Auth	`@fastify/jwt` + `@fastify/oauth2`	—	Discord OAuth → JWT HTTP-Only-Cookie

Frontend

LAYER	LIBRARY	VERSION	NOTIZ
Build	`Vite`	6	HMR, Proxy für `/api`, `/auth`, `/socket.io`
UI-Framework	`React`	19	Hooks, Concurrent
Router	`TanStack Router`	—	code-based (kein File-Routing)
Data-Fetching	`TanStack Query`	—	Query-Keys-Konvention: `[domain, ...params]`
Styling	`Tailwind CSS`	4	CSS-first via `@tailwindcss/vite`, kein `tailwind.config.js`
Drag & Drop	`@dnd-kit`	—	Im Preset-Editor
Bracket-Zoom	`react-zoom-pan-pinch`	—	Overlay-Buttons bei > 32 Players

Infra & Tests

BEREICH	TOOL	NOTIZ
Datenbank	`PostgreSQL 16`	via docker-compose lokal; CI: Service-Container
Cache	`Redis 7`	via docker-compose lokal; CI: Service-Container
Unit-Tests	`Vitest`	Backend `pool: 'forks', singleFork: true`
E2E	`Playwright (Chromium)`	Chromium-only, `baseURL: localhost:5173`
CI	`GitHub Actions`	lint → typecheck → unit → build → E2E
Scraper	`axios + cheerio + commander + winston`	Standalone-Package `@tww3/scraper`

03 Monorepo-Struktur & Architektur

> pnpm-Workspaces mit Turborepo. Klare Trennung Apps / Packages / Tools. Shared-Types als Single Source of Truth.

Repo-Tree

```
warhammer/ ├ apps/ │ ├ backend/ # @tww3/backend — Fastify 5 + Prisma 7 + Socket.IO │ ├ frontend/ # @tww3/frontend — Vite 6 + React 19 + Tailwind 4 │ └ e2e/ # @tww3/e2e — Playwright (Chromium) ├ packages/ │ ├ db/ # @tww3/db — Prisma-Schema + Migrations + Seed + Singleton │ └ types/ # @tww3/types — Zod-Schemas + Socket-Event-Interfaces ├ scraper/ # @tww3/scraper — CLI-Tool für totaltavern.com ├ .knowledge/ # 11 dichte Topic-Files für Sub-Agents ├ .github/workflows/ # ci.yml — Lint, Typecheck, Test, Build, E2E ├ docker-compose.yml # Postgres 16 + Redis 7 ├ pnpm-workspace.yaml ├ turbo.json ├ CLAUDE.md # auto-loaded Hub-Index └ ROADMAP.md # Milestone-SSOT
```

Apps & Packages

APP

@tww3/backend

Fastify-5-App mit `buildApp()`-Factory. Plugin-System (db → redis → auth → draft → socket → cron → routes). REST + GraphQL. JWT in HTTP-Only-Cookie.

APP

@tww3/frontend

Vite-6-SPA. TanStack Router (code-based), TanStack Query, Tailwind 4 CSS-first. `apiFetch<T>()` als zentraler API-Wrapper.

APP

@tww3/e2e

Playwright-Chromium E2E. 9 Specs inkl. live-draft (2 Tabs, echte WebSocket, ~38s) und reconnect-recovery (Redis-Rehydrate).

PACKAGE

@tww3/db

Prisma-7-Schema mit 20 Models, 4 Migrations, Seed-Script, Singleton-Client. `datasource.url` in `prisma.config.ts`, nicht in `schema.prisma`.

PACKAGE

@tww3/types

Single Source of Truth: `api-schemas.ts` (Zod), `socket-events.ts` (Server/Client-Interfaces), `draft.ts`, `bracket.ts`.

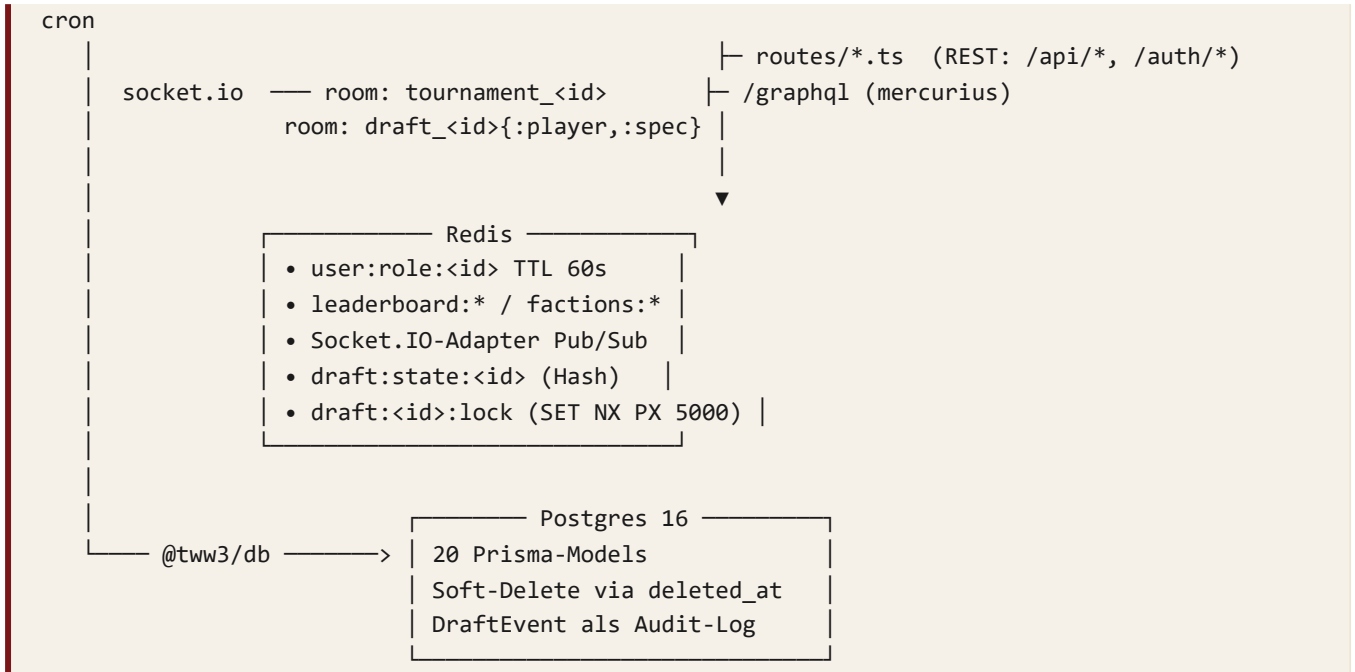
TOOL

@tww3/scraper

CLI mit commander/axios/cheerio. Rate-Limit 2000 ms. `ImportLog`-Tabelle für Audit. CI-Guard via `process.env.CI`.

Datenfluss-Skizze

```
Browser  ──(HTTP/WS via Vite-Proxy 5173)──>  Fastify 3000
   │                                              │
   │  cookie: auth_token (JWT, HTTP-Only)         ├─ plugins: db, redis, auth, draft, socket,
```


cron
socket.io
room: tournament_<id>
room: draft_<id>{:player,:spec}
routes/*.ts (REST: /api/*, /auth/*)
/graphql (mercurius)
Redis
• user:role:<id> TTL 60s
• leaderboard:* / factions:*
• Socket.IO-Adapter Pub/Sub
• draft:state:<id> (Hash)
• draft:<id>:lock (SET NX PX 5000)
@tww3/db
Postgres 16
20 Prisma-Models
Soft-Delete via deleted_at
DraftEvent als Audit-Log

04 Backend-Architektur

Plugin-Reihenfolge (kritisch)

Die Reihenfolge in `buildApp()` spiegelt Abhängigkeiten: untenstehende Plugins dekorieren `fastify.*`, die obere Plugins/Routes konsumieren.

#	PLUGIN	DEKORIERT	BEGRÜNDUNG
1	`db`	`fastify.prisma`	Fast jedes andere Plugin braucht Prisma
2	`redis`	`redis`, `redisPub`, `redisSub`	Auth-Cache, Socket-Adapter, Draft-State
3	`auth`	`authenticate`, `requireRole`, Cookie-Helper	Vor Routes (preHandler-Dekorator)
4	`draft`	`fastify.draftService`	Vor Socket (Socket-Handler ruft Service)
5	`socket`	`fastify.io`	Braucht auth + redis + draft
6	`cron`	—	Nach Routes registrierbar
7	`routes/*`	—	Konsumieren alle Dekoratoren

BuildAppOptions — Feature-Flags für Tests

```
const app = await buildApp({
  withSocket: false,   // skip Socket.IO
  withRedis:  false,   // cached() fällt graceful durch
  withCron:   false,   // kein node-cron-Trigger
  withGraphql:false,
  withDraft:  true,
});
```

REST-Route-Map (Auszug)

FILE	PREFIX	WICHTIGSTE ENDPOINTS
`auth.ts`	/auth	`/discord`, `/discord/callback`, `/logout`, `/test-login` (NODE_ENV=test)
`tournaments.ts`	/api/tournaments	CRUD, `POST :id/start` (Bracket-Generierung)
`participants.ts`	/api/tournaments/:slug	`POST /register`, `DELETE /withdraw`
`matches.ts`	/api/matches	`POST :id/result`, `PATCH :id/start`
`bracket.ts`	/api/tournaments/:slug	`GET /bracket`

FILE	PREFIX	WICHTIGSTE ENDPOINTS
`leaderboard.ts`	/api/leaderboard	Paginated mit `RANK() OVER` Tie-Break
`factions.ts / meta.ts`	/api/factions, /api/meta	Listing, Detail, Matchup-Heatmap
`drafts.ts / draft-presets.ts`	/api/drafts, /api/draft-presets	Draft-Lifecycle, CRUD + Promote
`admin.ts`	/api/admin	`audit-log`, `stats`, `ban/unban` (ADMIN-only)
`army-lists.ts`	/api/army-lists	Multipart-Upload (TXT/PDF, ≤ 5 MB)

Caching-Pattern

```
const data = await cached(
  fastify.redis,
  cacheKey('leaderboard:season', { seasonId, page }),
  () => fetchFromDb(),
  { ttlSeconds: 60 }
);
// Bei Mutation: invalidate(fastify.redis, 'leaderboard:*');
```

Read-Through. Bei `fastify.redis === undefined` (Test-Mode) fällt `cached()` graceful durch zu `compute()`.

05 Frontend & Datenbank

Frontend — Code-based Router

Alle Routes in `apps/frontend/src/router.tsx` via `createRoute()`. Kein File-based Routing. 14 Top-Level-Routes:

- `/` → IndexPage
- `/login` → LoginPage
- `/tournaments/create` → CreateTournamentPage
- `/tournaments/$slug` → TournamentDetail
- `/leaderboard` → LeaderboardPage
- `/users/$id` → UserProfilePage
- `/meta` → MetaDashboard
- `/factions` + `/factions/$id`
- `/drafts/$id` → DraftLobbyPage
- `/drafts/$id/spectate` → DraftSpectatorPage
- `/presets` + `/presets/new` + `/presets/$id/edit`
- `/admin` → AdminPage (4 Tabs)

API-Layer

```
// apps/frontend/src/lib/api.ts
const data = useQuery({
  queryKey: ['leaderboard', seasonId, page],
  queryFn: () => apiFetch<LeaderboardResponse>(
    `/api/leaderboard?seasonId=${seasonId}&page=${page}`
  ),
});
```

`apiFetch<T>()` fügt automatisch `credentials: 'include'` hinzu und wirft bei Non-2xx. Query-Key-Konvention: `[domain, ...params]`.

Datenbank — 20 Prisma-Models

DOMAIN	MODELS
User & Auth	`User`, `AuditLog`
Tournament-Core	`Season`, `Tournament`, `TournamentParticipant`, `TournamentFactionAllowlist`, `Match`, `TournamentResult`
Teams	`Team`, `TeamMember`
Stats & Leaderboard	`LeaderboardEntry`, `Faction`, `FactionStats`, `FactionStatsSnapshot`, `MatchupStats`
Draft	`DraftPreset`, `Draft`, `DraftEvent` (append-only Audit)

DOMAIN	MODELS
Army-Listen	`ArmyList`
Scraper	`ImportLog`

Realtime — Socket.IO

ROOM	ZWECK
`tournament_<id>`	Live-Bracket-Updates pro Turnier (Match-Results, Status-Changes)
`draft_<id>`	Player-Room — sieht alles inkl. hidden
`draft_<id>:player_<userId>`	Per-Player-Room für persönliche Sicht
`draft_<id>:spec`	Spectator-Room mit hidden-Maskierung

Draft-System — Architektur-Highlight

Trennung von **Pure State-Machine** (`lib/draft-state.ts` , kein I/O, voll testbar) und **I/O-Layer** (`lib/draft-service.ts` mit Prisma + Redis + Socket). Hybrid-State-Store: Redis-Hash als hot state + `Draft.state` JSON als Source-of-Truth + `DraftEvent` -Tabelle als append-only Audit. Redis-Lock via `SET NX PX 5000` verhindert Race-Conditions bei parallelen Commits.

06 Milestones M1 – M5

> Greenfield 2026-05-12 → Production-Launch-Ready 2026-05-13. Fünf Milestones in zwei Tagen, AI-first Solo-Dev mit Plan→Execute-Workflow (Opus-Plan, Sonnet-Wellen).

M1 — Foundation & Live-Core DONE

Monorepo-Bootstrap, Prisma-Schema initial, Discord-OAuth-Flow, JWT-Cookie-Auth, Single-Elimination-Bracket-Generierung, Match-Result-Reporting, Live-Bracket via Socket.IO (Room `tournament_<id>`), Vite-React-Setup mit TanStack Router, BracketView-Component mit Custom-SVG.

M2 — Swiss-System & Leaderboard DONE

Swiss-Paarungs-Logik mit Rematch-Avoidance, Round-Robin-Format, Leaderboard mit Season-System, Tournament-Points pro Format, finalize-tournament-Hook beim letzten `COMPLETED`-Match, UserProfile-Page.

M3 — Faction-Stats, ELO & Heatmap DONE

ELO-Algorithmus (Multi-Player Performance-Rating, K=32 normal / K=48 Major, zero-sum), `MatchupStats`-Counter im Match-Result-Hook mit `faction_a < faction_b`-Konvention, 24×24-Matchup-Heatmap via `$queryRaw`, REST + GraphQL für Faction/Meta-Endpoints, täglicher `FactionStatsSnapshot` via node-cron 00:05 UTC, `requireRole` mit Redis-Cache (`user:role:<id>` TTL 60s).

M4 — Captain's-Mode-Draft DONE

Pure State-Machine in `lib/draft-state.ts` (8 Top-Level-State-Keys inkl. hidden_pick/ban_variants), `DraftService`-Klasse als I/O-Wrapper, Redis-Lock + Hybrid-State-Store, Timer pro Preset einstellbar (Default 30s) mit Auto-Random bei DC, drei Socket-Rooms (player/per-player/spectator) mit `maskStateForViewer()`, dedizierte `DraftEvent`-Tabelle, voller UI-Preset-Editor mit @dnd-kit, zwei Seed-Presets ("Standard 1v1", "Captain's Mode Classic").

M5 — Polish, Scraper, Admin & E2E DONE

M5.1 Foundation: Test-Login-Bypass (NODE_ENV=test), `RANK() OVER`-Tie-Break im Leaderboard, hermetic-Test-Fixtures mit randomUUID-Cleanup, Playwright in CI.
M5.2 E2E-Suite: 5 kritische Journeys — happy-path 16-Player, live-draft 2 Tabs ~38s, swiss-rematch-avoidance, leaderboard-correctness, reconnect-recovery mit Redis-Rehydrate.
M5.3 Scraper: Standalone-Package mit commander/cheerio/axios, 2000 ms Rate-Limit, `ImportLog`-Audit, CI-Guard.
M5.4 Admin + Army-List: Audit-Log-Pagination, Stats-Dashboard (60s-Cache), Ban/Unban via `deleted_at`, Preset-Promotion (`PATCH :id/promote`), Multipart-Upload (TXT/PDF ≤ 5 MB) mit naivem Parser für Lord/Heroes/Units.
M5.5 Polish: Bracket-Zoom-Overlay bei > 32 Players, Status-Tailwind-Farben, `lib/timezone.ts` ersetzt 6 `toLocale*`-Calls, Mobile-Hamburger-Drawer, OpenGraph + Twitter-Card, sitemap.xml + robots.txt, 25 Faction-Icon-SVG-Placeholders, production-smoke.spec.ts, DEPLOYMENT.md.

Tests final M5-Ende

301	60	23+	12

BACKEND

FRONTEND

E2E

SCRAPER

07 Patterns, Gotchas & Knowledge-Hub

Kritische Gotchas

Prisma 7 — `datasource.url` in `prisma.config.ts`

Mit dem `driver-adapter`-Setup gehört die DB-URL **nicht** in `schema.prisma`, sondern in `packages/db/prisma.config.ts`. `schema.prisma` enthält nur den `provider`. Andernfalls: silent fail bei Migrationen.

`fastify-socket.io` ist Fastify-4-only

Auf Fastify 5 funktioniert das offizielle Plugin nicht. Workaround: Socket.IO direkt an `fastify.server` attachen: `new IOServer(fastify.server, ...)` in `apps/backend/src/plugins/socket.ts`, dann Redis-Adapter via `io.adapter(createAdapter(redisPub, redisSub))`.

ESM-Imports brauchen `.js`-Extension

Auch in TypeScript-Files: `import {{ cached }} from './cache.js'` — nicht `'./cache'`. TypeScript löst das zu `.ts` auf, der ESM-Loader braucht `.js` zur Laufzeit.

Auth-Hook-Reihenfolge

`fastify.authenticate` MUSS vor `fastify.requireRole(...)` registriert sein — sonst ist `request.user` nicht gesetzt, wenn `requireRole` die Rolle prüft.

Hermetic Test-Cleanup

Kein globales `prisma.*.deleteMany()` in Tests — würde Seed-Daten löschen. Stattdessen randomUUID-Namen in Fixtures und gezieltes Cleanup pro Test-Lauf via `cleanupUsers(userIds)` aus `test/helpers/db-fixtures.ts`.

Knowledge-Hub (neu, Commit 084350d)

Sub-Agents (Explore, general-purpose, Plan) haben kein Memory. Damit sie nicht jedes Mal den Stack re-discovern müssen, gibt es seit 2026-05-13 einen dichten `.knowledge/`-Hub:

stack.md

Workspaces, Commands, Gotchas

backend-architecture.md

Plugin-Reihenfolge, Routes, lib/

database.md

20 Models, Migrations, Setup

caching.md

auth.md

realtime.md

cached/invalidate/cacheKey API

Discord-OAuth, JWT, requireRole

Socket.IO, Rooms, Events

draft-system.md

State-Machine, Lock, Maskierung

frontend-patterns.md

Router, Query-Keys, apiFetch

testing.md

Vitest-Config, Fixtures, Playwright

algorithms.md

ELO, Bracket, Swiss, Cron

types-contracts.md

@tww3/types exports

+ 4× CLAUDE.md

Root (auto-loaded) + apps/{backend,frontend,e2e}

Sub-Agent-Konvention: Prompts beginnen mit `Lies zuerst .knowledge/X.md`, statt Source-Code-Re-Discovery. Verification-Test (Caching/Socket/Auth-Fragen) → PASS mit Confidence HOCH, ohne Source-Reads.

08 Production-Launch-TODOs & Appendix

Pre-Launch-TODOs (in DEPLOYMENT.md)

- **og-image.png** durch echtes 1200×630 PNG ersetzen (aktuell Placeholder)
- **example.com** in `sitemap.xml` / `robots.txt` durch echte Domain ersetzen
- **SVG-Faction-Icons** durch echte Artworks tauschen (Lizenz vom User vorhanden)
- **Scraper-Selektoren** beim ersten Live-Lauf gegen totaltavern.com tunen
- **smoke.spec.ts:15** Discord-Button-Test fixen (pre-existing seit M5)

Häufig genutzte Commands

```
# Dev
pnpm dev                          # alle Apps via Turborepo
pnpm -F @tww3/backend dev         # nur Backend
pnpm -F @tww3/frontend dev        # nur Frontend

# Tests
pnpm test                         # Backend + Frontend Unit
pnpm test:e2e                     # Playwright (Chromium)
pnpm -F @tww3/backend test -- <pattern>

# Database
pnpm db:migrate                   # prisma migrate dev
pnpm db:migrate:deploy            # CI/Prod
pnpm db:generate                  # prisma generate
pnpm db:seed                      # seed.ts
pnpm db:studio                    # Port 51212

# Infra
pnpm docker:up / docker:down      # Postgres 16 + Redis 7
```

Key-Paths

PFAD	ZWECK
`apps/backend/src/app.ts`	`buildApp()`-Factory, Plugin- & Route-Registrierung
`apps/backend/src/lib/cache.ts`	`cached` / `invalidate` / `cacheKey`
`apps/backend/src/lib/draft-state.ts`	Pure Draft-State-Machine
`apps/backend/src/lib/draft-service.ts`	DraftService (Redis + Prisma + Timer + Socket)
`apps/backend/src/plugins/socket.ts`	Socket.IO direkt an `fastify.server`
`packages/db/prisma/schema.prisma`	20 Models

PFAD	ZWECK
`packages/db/prisma.config.ts`	Prisma-7 driver-adapter (DB-URL hier)
`packages/types/src/socket-events.ts`	Socket-Event-Interfaces
`apps/frontend/src/router.tsx`	Code-based TanStack Router
`apps/frontend/src/lib/api.ts`	`apiFetch<T>()` Wrapper
`apps/e2e/tests/helpers/tournament-fixture.ts`	E2E-Setup-Helpers
`.knowledge/`	11 dichte Topic-Files für Sub-Agents
`CLAUDE.md`	Auto-loaded Hub-Index
`ROADMAP.md`	Milestone-SSOT
`DEPLOYMENT.md`	Production-Setup & Pre-Launch-TODOs

Repository & Commit-Stand

COMMIT	BESCHREIBUNG
`084350d`	**HEAD** · docs: .knowledge hub + CLAUDE.md scaffolding
`3ba8f1c`	feat: M5.5 polish + SEO + launch-prep
`42a176e`	feat: M5.4 admin panel + army-list upload
`a1c0c7d`	feat: M5.3 scraper package
`bf4e36a`	feat: M5.2 E2E suite (5 critical journeys)
`7d60a78`	feat: M5.1 foundation & hardening

Dokument generiert am 2026-05-13 für Solo-Dev-Workflow. Quellen: Repository-Code, ROADMAP.md, DEPLOYMENT.md, .knowledge/-Hub. Kein Anspruch auf Vollständigkeit der Spec — primäre SSOTs bleiben Repo + ROADMAP.md.